

Deploying on Amazon Web Services

How to host the Endpoint Intelligence server on AWS: EC2, security groups, a permanent service, authentication, HTTPS, and managed upgrades. The agent on each laptop is unchanged — only the server moves to the cloud.

EC2 + Elastic IP + systemd • API-key auth • HTTPS • optional RDS / Fargate

1. What changes when you move to AWS

Only the **server** moves. The agent still runs on every laptop and still sends telemetry to a URL — that URL just points at AWS instead of a machine on your LAN. Two things become important that a private network let you ignore:

- **A public, stable endpoint.** Agents reach the server over the internet, so it needs a fixed public address (an Elastic IP) or a domain name.
- **Security you did not need internally.** On a LAN, only local machines could reach the server. On the internet, the telemetry endpoint must be authenticated and traffic must be encrypted (HTTPS), or anyone could send fake data or read machine details in transit.

The agent code barely changes

The only agent change in this whole guide is adding one authentication header (section 6). Everything else is AWS configuration around the same server you already built.

Recommended starting architecture

```
Laptops (agents)
  | HTTPS + API key, over the internet
  v
[ Elastic IP ] --> EC2 instance (Ubuntu)
                  |-- Caddy (HTTPS / TLS termination, port 443)
                  |-- uvicorn (server, localhost:8000, via systemd)
                  |-- SQLite (or RDS PostgreSQL, see section 8)
                  Security Group = cloud firewall
```

Start small, upgrade only when needed

A single EC2 instance with SQLite handles a modest fleet comfortably. RDS, load balancers, and containers (sections 8–9) are upgrades to add later if the fleet grows — do not build them up front.

2. Before you start

- An AWS account with permission to create EC2, Security Groups, and Elastic IPs.
- An SSH key pair (create one during instance launch if you do not have one).
- The `server/` and `ml/` folders ready to copy up (via `scp` or a private git repo).
- For HTTPS: a domain or subdomain you can point at the server, e.g. `health.yourcompany.com` (section 7 explains why).

Decide who should reach the server

If your laptops connect from known office networks, restrict access to those IP ranges in the Security Group rather than opening the server to the whole internet. If laptops are fully remote, you will open it publicly but rely on the API key and HTTPS — or use a VPN (section 10) for the tightest security.

3. Launch the EC2 instance

- In the EC2 console, choose **Launch instance**.
- **AMI:** Ubuntu Server (22.04 or 24.04 LTS).
- **Instance type:** `t3.small` is a sensible start (2 vCPU, 2 GB). Scale up later if needed.
- **Key pair:** select or create one; you will use it to SSH in.
- **Storage:** the default 8–20 GB is fine for SQLite and logs; increase if you expect a lot of history on one box.
- **Security group:** create a new one — configured next.

4. Configure the Security Group

The Security Group is your cloud firewall — it replaces the Windows Firewall rule from the on-prem guide. Add only these inbound rules:

Type	Port	Source	Why
SSH	22	Your admin IP only	Manage the server; never open to all
HTTPS	443	Fleet IPs or 0.0.0.0/0	Agents send telemetry (via Caddy)
HTTP	80	Fleet IPs or 0.0.0.0/0	Lets Caddy obtain the TLS cert

Do not expose port 8000 or SSH to the world

The server (uvicorn on 8000) should listen only on localhost; Caddy on 443 is the only thing the internet talks to. Restrict SSH (22) to your own IP address — an open SSH port is a magnet for attacks.

Assign an Elastic IP

Allocate an Elastic IP and associate it with the instance. This gives the server a fixed public address that survives reboots — so your agents' `SERVER_URL` never has to change. Without it, a stopped instance gets a new IP and every agent loses the server.

5. Install the server as a permanent service

SSH into the instance, then:

```
sudo apt update && sudo apt install -y python3-pip python3-venv
# copy your server/ and ml/ folders to /home/ubuntu/ (scp or git clone)
cd /home/ubuntu/server
python3 -m venv venv
source venv/bin/activate
pip install fastapi uvicorn jinja2 requests
```

Create a systemd service

systemd keeps the server running across reboots and restarts it if it crashes — this solves the 'make the server permanent' problem cleanly. Create `/etc/systemd/system/endpoint-server.service`:

```
[Unit]
Description=Endpoint Intelligence Server
After=network.target

[Service]
User=ubuntu
WorkingDirectory=/home/ubuntu/server
Environment=SERVER_API_KEY=choose-a-long-random-secret
ExecStart=/home/ubuntu/server/venv/bin/uvicorn main:app \
    --host 127.0.0.1 --port 8000
Restart=always

[Install]
WantedBy=multi-user.target

sudo systemctl daemon-reload
sudo systemctl enable --now endpoint-server
sudo systemctl status endpoint-server    # confirm it is active
```

Why bind to 127.0.0.1, not 0.0.0.0

On AWS the server listens only on localhost; Caddy (next section) accepts public HTTPS and forwards to it. This means the raw, unauthenticated-looking port is never exposed — only the encrypted, front-door path is reachable.

6. Add API-key authentication

Once the server is reachable from the internet, the telemetry endpoint must reject anonymous requests. A shared secret is the simplest effective control: agents send it in a header, the server checks it. Three small code additions.

6.1 Server — check the key (main.py)

```
import os
from fastapi import Header, Depends, HTTPException

SERVER_API_KEY = os.environ.get('SERVER_API_KEY')

def require_key(x_api_key: str = Header(None)):
    # If a key is configured, requests must present a matching one.
    if SERVER_API_KEY and x_api_key != SERVER_API_KEY:
        raise HTTPException(status_code=401, detail='invalid API key')

# add the dependency to the telemetry endpoint:
@app.post('/telemetry')
def receive_telemetry(snapshot: dict, _=Depends(require_key)):
    ... # body unchanged
```

6.2 Agent — send the key (sender.py)

```
import os
API_KEY = os.environ.get('AGENT_API_KEY', '')

# in send(), add the header:
requests.post(TELEMETRY_URL, json=snapshot,
              headers={'X-API-Key': API_KEY}, timeout=REQUEST_TIMEOUT)
```

6.3 Set the same secret on both sides

- Server: the SERVER_API_KEY in the systemd file (section 5).
- Agents: set AGENT_API_KEY to the same value — add it to the scheduled-task environment, or store it in config and read it in sender .py.

The key is a shared secret — treat it as one

Use a long random string. Keep it out of source control. If it ever leaks, rotate it (change it on the server and re-push to agents). For stronger security later, move to per-agent keys or mutual TLS.

7. Enable HTTPS with Caddy

Telemetry includes machine serials and hardware details, so it should never travel as plain HTTP over the internet. Caddy is the simplest way to add HTTPS: it obtains and renews a free TLS certificate automatically and forwards traffic to the server.

Point a domain at the server

Create a DNS **A record** — e.g. `health.yourcompany.com` → your Elastic IP. Caddy needs a real domain (not a bare IP) to get a certificate. If you use Route 53 for DNS this is a one-record change; any DNS provider works.

Install and configure Caddy

```
sudo apt install -y debian-keyring debian-archive-keyring apt-transport-https
# (follow caddyserver.com/docs/install for the current apt steps)
sudo apt install -y caddy
```

Set `/etc/caddy/Caddyfile` to:

```
health.yourcompany.com {
    reverse_proxy 127.0.0.1:8000
}
```

```
sudo systemctl reload caddy
```

That is all: Caddy fetches a certificate, serves HTTPS on 443, and proxies to the server on `localhost:8000`. Agents now use `https://health.yourcompany.com` as their `SERVER_URL`.

Managed alternative: ALB + ACM

If you prefer AWS-native pieces, put an Application Load Balancer in front with a certificate from AWS Certificate Manager (ACM). It also enables running multiple server copies behind one address as the fleet grows. More moving parts than Caddy, so reach for it when you need the scale.

8. Point the agents at AWS

Update `agent/config.py` for the fleet:

```
MODE = "server"
SERVER_URL = "https://health.yourcompany.com" # your domain, HTTPS
```

Ensure each agent has the `AGENT_API_KEY` set (section 6.3), then deploy exactly as in the on-prem guide — the installer and scheduled task are identical. Confirm a test laptop appears on the dashboard at your domain before rolling out widely.

9. Managed upgrades (add later, if needed)

Upgrade	AWS service	When it is worth it
Managed database	RDS PostgreSQL (or Aurora)	When SQLite strains under many concurrent agents. This is the planned PostgreSQL upgrade.
No servers to manage	ECS on Fargate + ECR	Package the server as a Docker image and run it serverless. Best once the fleet is large.
Auto-scaling + HTTPS	Application Load Balancer	Run several server copies behind one address; ACM handles certificates.
Secret storage	AWS Secrets Manager	Hold the API key, DB password, and copilot key instead of env vars.
Logs and alarms	CloudWatch	Central server logs; alert if the server or a metric goes wrong.
Cloud LLM copilot	Amazon Bedrock	Run the copilot's model in AWS instead of local Ollama, if you prefer managed LLM.

10. Tightest security option: keep it private

If the laptops can reach AWS over a VPN (AWS Client VPN or Site-to-Site VPN), you can keep the server off the public internet entirely — agents reach it on a private address, and the Security Group allows no public inbound at all. This is the most secure design; the trade-off is VPN setup and that fully-remote laptops must be on the VPN to report. Even then, keep the API key as defence in depth.

11. Rough cost picture

Approximate on-demand figures for a small deployment — always check current AWS pricing for your region, as prices change and reserved or savings plans reduce them significantly.

Component	Rough monthly cost
t3.small EC2 instance (on-demand, always on)	~ \$15
Elastic IP (while attached to a running instance)	no charge
Storage (EBS, ~20 GB)	~ \$2
Data transfer (telemetry is small)	minimal
Optional: RDS db.t3.micro/small PostgreSQL	~ \$15–\$30

Cut cost with reserved capacity

For an always-on server, a 1-year Reserved Instance or Savings Plan cuts the EC2 cost substantially versus on-demand. A single small instance is usually the largest line item.

12. Go-live checklist (AWS)

#	Item
1	EC2 instance launched; Elastic IP allocated and associated
2	Security Group: SSH restricted to your IP; 443/80 as intended
3	Server installed and running under systemd (survives reboot)
4	SERVER_API_KEY set on the server; AGENT_API_KEY set on agents

5	DNS A record points your domain at the Elastic IP
6	Caddy serving HTTPS and proxying to the server
7	config.py uses the HTTPS domain as SERVER_URL
8	A test laptop reports in and appears on the dashboard
9	Server bound to 127.0.0.1 (not 0.0.0.0); port 8000 not public
10	Backups/plan for the database (or RDS with automated backups)

Recommended first build: one EC2 instance + Elastic IP + systemd + Security Group, with API-key auth and Caddy HTTPS, keeping SQLite until the fleet size demands RDS. Smallest jump from the on-prem setup, fully internet-reachable and secured.